

Microservices Architecture Guide

ChatAssist Platform Architecture, Service Catalog & Patterns | Version 3.0

1. Architecture Overview and Design Principles

TechViet's microservices architecture evolved from a monolithic Django application that served as the foundation for the ChatAssist product from 2019 to 2023. The decision to decompose the monolith was made in Q2 2023 after a series of incidents where deployments to one feature (e.g., the analytics dashboard) caused cascading failures in unrelated features (e.g., the real-time chat engine). The decomposition followed the Strangler Fig pattern, where new features were built as separate services and existing features were gradually extracted from the monolith over a 12-month period. Today, the ChatAssist platform consists of 7 core microservices, each owned by a specific team and deployable independently. Our architecture adheres to the following design principles, established by the Architecture Review Board (ARB) in consultation with the CTO: first, services must be organized around business capabilities rather than technical layers — for example, we have a 'Conversation Service' rather than a 'Database Service' and a 'Compute Service'. Second, each service owns its data — there are no shared databases, and cross-service data access happens exclusively through published APIs or asynchronous events. This eliminates the tight coupling that plagued the monolith and allows each team to choose the most appropriate data store for their workload. Third, all inter-service communication must be resilient to failure — every service call includes circuit breakers, timeouts, and retry logic with exponential backoff. Fourth, services must be observable — every service emits structured logs, request traces (using OpenTelemetry), and business metrics. Fifth, backward compatibility must be maintained for at least two versions of any API — breaking changes require a deprecation period of at least 30 days with explicit communication to all consuming teams.

2. Service Catalog

2.1 Conversation Service (conversation-svc)

The Conversation Service is the core of the ChatAssist platform, responsible for managing chat conversations between end-users and AI assistants. It handles conversation lifecycle (create, continue, archive), message persistence, and real-time message delivery via WebSocket connections. The service is implemented in Go (version 1.22) for optimal performance in handling concurrent WebSocket connections, using the gorilla/websocket library for WebSocket management and gRPC for synchronous inter-service communication. Data store: PostgreSQL 15 (primary conversation data: messages, threads, participants) with a Redis 7 cluster for active session state and message delivery queues. The PostgreSQL database uses partitioning by conversation creation date (monthly partitions) to maintain query performance as the data volume grows — we currently have approximately 450 million messages across 12 million conversations, growing at approximately 2 million messages per day. The Redis cluster is configured with 3 shards and 2 replicas per shard to handle the WebSocket session state for up to 50,000 concurrent connections. Key API endpoints: POST /v2/conversations (create new conversation), GET /v2/conversations/{id}/messages (retrieve message history with cursor-based pagination), WS /v2/conversations/{id}/stream (WebSocket endpoint for real-time message delivery), POST

/v2/conversations/{id}/messages (send a new message, which triggers the AI pipeline), and DELETE /v2/conversations/{id} (soft-delete a conversation). The service publishes events to the Kafka topic 'conversation.events' for: conversation.created, message.sent, message.received, conversation.archived. These events are consumed by the Analytics Service, Notification Service, and the ML Pipeline for model training data collection. Performance SLOs: p50 latency for message send < 50ms, p99 latency for message send < 200ms, WebSocket connection establishment < 100ms, message delivery latency (send to receive) < 300ms. Current performance as of Q4 2025: p50 = 32ms, p99 = 145ms, WebSocket establishment = 67ms, delivery latency = 180ms.

2.2 AI Engine Service (ai-engine-svc)

The AI Engine Service orchestrates the AI/ML pipeline that powers ChatAssist's intelligent responses. When a user sends a message, the Conversation Service forwards it to the AI Engine via gRPC, which then executes the following pipeline: intent classification (determines whether the message is a question, command, or chitchat), entity extraction (identifies key entities like dates, names, product references), knowledge retrieval (searches the organization's knowledge base using hybrid search combining BM25 and dense vector retrieval), response generation (uses a fine-tuned large language model to generate a contextually appropriate response), and safety filtering (checks the generated response against content policy rules and PII detection). The service is implemented in Python 3.12 using FastAPI for the HTTP API and grpcio for the gRPC interface. ML models are served using NVIDIA Triton Inference Server running on the GPU nodes described in the infrastructure documentation. The primary language model is a fine-tuned Llama 3.1 8B model, quantized to INT8 for inference efficiency, which achieves 95% of the quality of the full-precision model at 3x the throughput. The knowledge retrieval component uses a custom embedding model (based on multilingual-e5-large, fine-tuned on Vietnamese customer support data) with Elasticsearch as the vector store. The AI Engine is designed for horizontal scalability: each instance is stateless and can process requests independently. The gRPC load balancing uses client-side round-robin with health checking, ensuring even distribution of requests across instances. GPU utilization is monitored continuously, and the Kubernetes Horizontal Pod Autoscaler (HPA) scales the deployment based on a custom metric: average GPU utilization across all pods, with a target of 70% utilization. The current production deployment runs 4 pods on the GPU node group, each with 1 NVIDIA A10G GPU allocated via the NVIDIA device plugin. Performance SLOs: intent classification latency < 20ms (p99), knowledge retrieval latency < 100ms (p99), full pipeline latency (message received to response generated) < 2000ms (p99), model accuracy (measured by human evaluation) > 90% for in-domain queries. Current performance: intent classification = 12ms (p99), retrieval = 78ms (p99), full pipeline = 1450ms (p99), accuracy = 92.3% (Q4 2025 evaluation).

2.3 User Management Service (user-mgmt-svc)

The User Management Service handles authentication, authorization, user profiles, and organization management for the ChatAssist platform. It serves as the identity provider for all ChatAssist services and implements OAuth 2.0 / OpenID Connect (OIDC) for authentication with support for multiple identity providers through SAML 2.0 federation (for enterprise SSO customers) and social login (Google, Microsoft). The service is implemented in Java 21 using Spring Boot 3.2, chosen for its mature security ecosystem (Spring Security) and the team's deep expertise in the JVM ecosystem. Data is stored in PostgreSQL 15 with a dedicated database (not shared with other services, per our architecture principles). User passwords are hashed using Argon2id with the following parameters: memory=65536 KB, iterations=3, parallelism=4, which provides strong resistance against both GPU-based and ASIC-based cracking attacks. Session tokens are JWTs signed with RS256 (RSA 2048-bit keys), with a short expiry of 15 minutes and refresh tokens valid for 7 days. The refresh token rotation pattern ensures that each refresh token can only be used once; if a refresh token is replayed, the entire token family is revoked as a security measure against token theft. Organization management is a key feature for enterprise customers.

Each organization can configure: SSO integration with their corporate IdP, custom roles and permissions beyond the default set (Admin, Agent, Supervisor, Viewer), data retention policies, IP allowlisting for API access, and audit logging with export to their SIEM. The multi-tenancy model uses row-level security in PostgreSQL, where every table includes an `org_id` column and all queries are automatically filtered by the current user's organization. This approach was chosen over schema-per-tenant (too many schemas at scale) and database-per-tenant (operational overhead and inability to run cross-tenant analytics).

2.4 Analytics Service (analytics-svc)

The Analytics Service processes conversation events from Kafka to generate real-time dashboards and scheduled reports for ChatAssist customers. It provides metrics such as: conversation volume (total, by channel, by hour), response time distribution (time from user message to AI response), resolution rate (percentage of conversations marked as resolved without human escalation), customer satisfaction scores (derived from post-conversation surveys and implicit signals like conversation length and re-contact rate), and agent performance metrics (for hybrid AI+human deployments: handle time, quality scores, escalation rate). The service uses a lambda architecture with two processing paths: a real-time path using Apache Flink for streaming aggregations (providing dashboard updates within 5 seconds of an event), and a batch path using Apache Spark for daily/weekly/monthly aggregation jobs that produce the scheduled reports. Both paths write to ClickHouse, an OLAP database optimized for analytical queries, which serves as the query engine for the dashboard API. ClickHouse was selected after evaluating Apache Druid, TimescaleDB, and BigQuery; the deciding factors were ClickHouse's superior query performance for our specific query patterns (time-series aggregations with high-cardinality dimensions like `customer_id` and `agent_id`) and its ability to run on-premises (the analytics data includes customer conversation metadata that falls under data residency requirements). The Flink streaming job consumes from multiple Kafka topics (`conversation.events`, `message.events`, `feedback.events`) and maintains stateful aggregations using RocksDB state backend. The state checkpoint interval is 60 seconds, written to the shared NFS volume. This means that in the event of a Flink job restart, the maximum data reprocessing window is 60 seconds. The Spark batch jobs are scheduled via Airflow and typically process the previous day's data starting at 02:00 UTC, completing by 04:00 UTC for Vietnam timezone (UTC+7) morning reports.

2.5 Notification Service (notification-svc)

The Notification Service manages all outbound communications from the ChatAssist platform, including email notifications, push notifications (iOS and Android), SMS alerts, and webhook deliveries for API integrations. The service is implemented in Node.js 20 using the NestJS framework, chosen for its event-driven architecture that aligns well with the notification delivery pattern (receive event, fan out to multiple channels). The delivery pipeline works as follows: notification events arrive from Kafka (e.g., 'new message in monitored conversation', 'SLA breach warning', 'daily summary ready'), the routing engine determines which channels to deliver to based on user preferences and organization notification policies, each channel adapter (email, push, SMS, webhook) formats the notification appropriately and delivers it, and delivery status (sent, delivered, failed, bounced) is tracked for each notification instance. Channel-specific implementation details: Email uses Amazon SES with dedicated IP addresses for deliverability, supporting both transactional emails (instant delivery with < 30 second SLA) and bulk emails (batched delivery for digests and reports). Email templates are stored in DynamoDB and support i18n with Vietnamese and English templates for all notification types. Push notifications use Firebase Cloud Messaging (FCM) for Android and Apple Push Notification Service (APNs) for iOS, with a unified interface that handles platform-specific payload formatting, badge counting, and silent notifications for background data sync. SMS is delivered through a local Vietnamese provider (FPT Telecom's SMS Gateway) for domestic numbers and Twilio for international numbers, with automatic fallback from the primary provider to the secondary if delivery fails. Webhook delivery implements a reliable delivery pattern with exponential backoff retry (5 attempts over 24 hours), payload signing using HMAC-SHA256 for verification, and a dead

letter queue for permanently failed deliveries that triggers an alert to the customer's admin.

3. Inter-Service Communication Patterns

Our microservices use three communication patterns, each chosen for specific use cases based on the requirements for latency, reliability, and coupling. Understanding these patterns is essential for any engineer working on the ChatAssist platform, as incorrect pattern selection is the most common source of reliability issues in distributed systems.

Pattern 1: Synchronous gRPC (for real-time request-response). Used when the caller needs an immediate response to proceed. Examples: Conversation Service calling AI Engine to get a response, User Management Service validating tokens for other services. All synchronous calls must implement: connection pooling (gRPC channels are reused, not created per-request), circuit breakers using the gobreaker library (Go services) or Resilience4j (Java services) with a failure threshold of 5 failures in a 60-second window, timeouts (default 5 seconds, configurable per-endpoint), and client-side retry with jittered exponential backoff (max 3 retries, initial delay 100ms). The circuit breaker state is exposed as a Prometheus metric and triggers a PagerDuty alert when any circuit opens.

Pattern 2: Asynchronous Events via Kafka (for decoupled, eventually consistent communication). Used when the producer does not need to wait for consumers to process the event. Examples: Conversation Service publishing conversation.created events consumed by Analytics and Notification services. Kafka is configured with: 3 brokers for high availability, replication factor of 3 for all production topics, message retention of 7 days (allowing consumers to replay events if needed), and Avro schema registry for schema evolution with backward compatibility enforcement. Consumer groups follow the naming convention '[service-name].[topic-name].consumer' for easy identification in monitoring dashboards.

Pattern 3: Synchronous HTTP REST (for external-facing APIs and simple internal queries). Used for the public API exposed to customers and for simple internal queries where gRPC's code generation overhead is not justified. All REST endpoints follow our API design standards (documented in TVSC-ENG-003) with JSON payloads, standard error response format, request ID propagation, and OpenAPI 3.1 specifications generated from code annotations.

4. Data Management and Consistency

In a microservices architecture where each service owns its data, maintaining data consistency across services is one of the most challenging aspects. We use the Saga pattern for distributed transactions that span multiple services, implemented using the choreography approach (event-driven sagas) rather than orchestration (centralized saga coordinator). The choreography approach was chosen because it avoids introducing a single point of failure and allows each service to evolve its saga steps independently. A concrete example is the 'New Enterprise Customer Onboarding' saga, which involves four services: User Management Service creates the organization and admin user, Conversation Service provisions the conversation workspace with default configuration, AI Engine Service creates the customer-specific knowledge base and indexes their initial documents, and Notification Service sends welcome emails and sets up notification channels. Each step publishes a completion event (e.g., 'organization.created', 'workspace.provisioned'), and the next service reacts to continue the saga. If any step fails, compensating events are published (e.g., 'organization.creation_failed'), and each preceding service executes its compensating action (e.g., deleting the partially created workspace). The saga's overall status is tracked in a dedicated 'saga_status' table in the initiating service's database, with a background job that detects stuck sagas (no progress event received within 5 minutes) and triggers either a retry or a full compensation. For read-heavy operations that need data from multiple services, we use the CQRS (Command Query Responsibility Segregation) pattern. The Analytics Service maintains materialized views that aggregate data from multiple services' event streams, providing fast query responses without requiring real-time cross-service calls. For example, the 'Conversation Dashboard' view combines data from conversation events (Conversation Service), user profile data (User Management events), AI quality

metrics (AI Engine events), and satisfaction scores (from the feedback events), all materialized into a ClickHouse table that can serve dashboard queries in under 100ms.